

Міністерство освіти і науки України
Національний технічний університет України «Київський
політехнічний інститут імені Ігоря Сікорського»

Звіт
з лабораторної роботи No 7
з дисципліни
«Мова програмування Java»

Виконала:
студентка групи ЗПІ-зп41
Ірина Левківська

Перевірив:
Орленко С.П.

Київ 2025

Поставка завдання (Завдання 8)

Розробити функцію, яка визначає кількість надпростих чисел в натуральному ряді. Функція приймає на вхід натуральне число не більше 1000. Надпростим називається число, якщо воно є простим та число, яке отримане з нього записом цифр у зворотному порядку, теж є простим.

Вирішення завдання

У цій задачі я реалізувала пошук надпростих чисел (їх також часто називають числами-емірпами, якщо вони не є паліндромами, але за умовою задачі ми рахуємо всі, що підпадають під правило "реверс теж простий").

Логіка реалізації:

- Алгоритм перевірки простоти (isPrime): Я використала класичний метод перевірки діленням до квадратного кореня числа n . Це дозволяє значно оптимізувати програму порівняно з перевіркою до самого числа n . Окремо оброблено парні числа та числа менші за 2.

- Реверс числа (reverseNumber): Замість того, щоб перетворювати число в рядок (що було б повільніше), я використала математичний підхід із залишком від ділення на 10 та множенням на 10. Це ефективний спосіб отримати число "задом наперед".

- Визначення надпростоти (isSuperPrime): Число вважається надпростим лише тоді, коли функція isPrime повертає true як для оригінального числа, так і для його реверсу.

```
1  import java.util.Scanner;
2
3  public class SuperPrimeFinder {
4      public static void main(String[] args) {
5          Scanner scanner = new Scanner(System.in);
6          System.out.print("Введіть натуральне число n (до 1000): ");
7
8          if (scanner.hasNextInt()) {
9              int n = scanner.nextInt();
10             if (n > 0 && n <= 1000) {
11                 int count = countSuperPrimes(n);
12                 System.out.println("Кількість надпростих чисел у ряді від 1 до " + n + ": "
13                                     + count);
14             } else {
15                 System.out.println("Будь ласка, введіть число в межах від 1 до 1000.");
16             }
17         } else {
18             System.out.println("Помилка: введено не ціле число.");
19         }
20     }
21 }
```

```

// Головна функція підрахунку надпростих чисел
public static int countSuperPrimes(int n) {
    int count = 0;
    for (int i = 1; i <= n; i++) {
        if (isSuperPrime(i)) {
            System.out.print(i + " "); // Виводимо знайдене число для наочності
            count++;
        }
    }
    System.out.println(); // Новий рядок після списку чисел
    return count;
}

// Перевірка, чи є число надпростим
public static boolean isSuperPrime(int number) {
    if (!isPrime(number)) {
        return false;
    }
    int reversed = reverseNumber(number);
    return isPrime(reversed);
}

// Метод для перевірки, чи є число простим
public static boolean isPrime(int n) {
    if (n < 2) return false;
    if (n == 2) return true;
    if (n % 2 == 0) return false;
    for (int i = 3; i <= Math.sqrt(n); i += 2) {
        if (n % i == 0) return false;
    }
    return true;
}

```

```

// Метод для запису цифр числа у зворотному порядку
public static int reverseNumber(int n) {
    int reversed = 0;
    while (n != 0) {
        int digit = n % 10;
        reversed = reversed * 10 + digit;
        n /= 10;
    }
    return reversed;
}

```

```

}

```

Скріншоти виконання коду

Введіть натуральне число n (до 1000): 10000
 Будь ласка, введіть число в межах від 1 до 1000.

```
Введіть натуральне число n (до 1000): 0
Будь ласка, введіть число в межах від 1 до 1000.
```

```
Введіть натуральне число n (до 1000): 3
2 3
Кількість надпростих чисел у ряді від 1 до 3: 2
```

```
Введіть натуральне число n (до 1000): 777
2 3 5 7 11 13 17 31 37 71 73 79 97 101 107 113 131 149 151 157 167 179 181 191 199 311 3
13 337 347 353 359 373 383 389 701 709 727 733 739 743 751 757 761 769
Кількість надпростих чисел у ряді від 1 до 777: 44
```

Висновок

Реалізація цієї функції допомогла мені відпрацювати навички розбиття складної задачі на декілька допоміжних методів. Використання математичних методів замість маніпуляцій з рядками дозволило забезпечити високу швидкість роботи програми навіть для максимального вхідного значення 1000.